

BCA152 Microcontrollers - Laboratory Activity No. 1
Real-Time Multisensor Room Monitoring System

Name: Abigil Queen Casas

Date: [Isulat ang petsa]

Subject/Section: [Isulat ang section]

Instructor: Asst. Prof. Paul Rodolf P. Castor, [M.Sc.](#)

1. Problem and Requirements

1.1 Problem Statement

This laboratory activity requires the design and implementation of a simulated ESP32-based room monitoring system using PlatformIO with the Espressif IoT Development Framework (ESP-IDF) and FreeRTOS. The system must monitor environmental conditions—temperature, humidity, ambient light, and motion—and provide real-time feedback through an OLED display, rotary encoder, and an alarm system.

1.2 Functional Requirements

The system implements the following functional requirements:

ID	Requirement	Implementation
FR-01	Temperature measurement	SensorTask reads DHT22 every 2 seconds
FR-02	Humidity measurement	SensorTask reads DHT22 every 2 seconds
FR-03	Ambient-light measurement	SensorTask reads LDR via ADC1_CHANNEL_6
FR-04	Motion detection	MotionTask monitors PIR on GPIO 26
FR-05	OLED display	DisplayTask owns and manages SSD1306
FR-06	Rotary encoder navigation	InputTask switches among 4 display modes
FR-07	Temperature alarm	AlarmTask activates buzzer outside 18°C–30°C
FR-08	Activity state	StateTask manages ACTIVE/INACTIVE
FR-09	Automatic inactivity	15-second timeout with no motion
FR-10	Automatic reactivation	PIR motion restores ACTIVE state

1.3 Wokwi Adaptation

The entire system was simulated in Wokwi using the following components: ESP32 DevKit C V4, DHT22, photoresistor (LDR), PIR motion sensor, KY-040 rotary encoder, SSD1306 OLED, and a passive buzzer. All GPIO pin assignments were verified against the Wokwi circuit diagram.

2. System Architecture and Design

2.1 Hardware Architecture

The system uses the ESP32 as the main microcontroller. The pin configuration is summarized below:

Component	ESP32 GPIO	Interface
DHT22 Data	GPIO 15	Single-wire
LDR Analog	GPIO 34	ADC1 Channel 6
PIR Out	GPIO 26	Digital Input
Encoder CLK	GPIO 18	Digital Input (Pull-up)
Encoder DT	GPIO 19	Digital Input (Pull-up)
OLED SDA	GPIO 21	I2C
OLED SCL	GPIO 22	I2C
Buzzer	GPIO 25	PWM (LEDC)

2.2 Software Architecture

The firmware is organized into multiple source modules, each with a single responsibility. The entire application is not placed in main.cpp; instead, it is decomposed into the following subsystems:

Sensors (sensors.cpp, dht22.cpp): DHT22 and LDR acquisition

Display (display.cpp, display_mode.cpp, oled_ssd1306.cpp): OLED driver and page logic

Input (input.cpp): Rotary encoder navigation

Motion (motion.cpp): PIR monitoring

Alarm (alarm.cpp, alarm_logic.cpp): Buzzer control and temperature decision logic

State (system_state.cpp, state_logic.cpp): ACTIVE/INACTIVE state machine

RTOS Objects (rtos_objects.cpp): Queue, mutex, and event group creation

2.3 Subsystem Decomposition

Hardware-independent decision logic (evaluateTemperature, evaluateSystemState, nextDisplayMode, previousDisplayMode) is separated from hardware drivers. This separation enables unit testing on the native platform without requiring an ESP32.

2.4 State Machine

The system operates in two primary states:

ACTIVE: OLED is enabled, sensor processing is normal, encoder is active, and the alarm is armed.

INACTIVE: OLED is turned off or blanked, unnecessary display operations are reduced, but motion detection remains operational. Any PIR trigger returns the system to ACTIVE.

The transition logic is implemented in evaluateSystemState() with a 15-second inactivity timeout.

3. FreeRTOS Architecture

3.1 Task Table

Task	Responsibility	Trigger / Period	Priority	Stack
MotionTask	Monitor PIR activity	Periodic (50 ms)	3	8192
InputTask	Process encoder navigation	Periodic (2 ms)	3	8192
StateTask	Manage ACTIVE/INACTIVE state	Periodic (100 ms)	2	8192
SensorTask	Read DHT22 and LDR	2 seconds (vTaskDelayUntil)	2	12288

AlarmTask	Evaluate alarm state and control buzzer	Event-driven	2	8192
DisplayTask	Own and manage the OLED	Event / Queue update	1	8192

Priority Justification:

MotionTask and InputTask are given priority 3 because they require prompt response to user input and motion events. Latency in these tasks would make the system feel unresponsive. StateTask, SensorTask, and AlarmTask are given priority 2 because they operate on a slower timescale (100 ms – 2 s) and can tolerate some latency. DisplayTask is given priority 1 because display updates are the least time-critical. It can be preempted by any other task without affecting system correctness.

3.2 Task States

During execution, each task transitions through the following states:

Running: The task is currently executing on the CPU.

Ready: The task is ready to run but is waiting for the scheduler to grant CPU time.

Blocked: The task is waiting for an event (e.g., queue data, delay expiration). This is the state of most tasks most of the time.

Suspended: Not used in this implementation.

Deleted: Not used in this implementation.

3.3 Inter-Task Communication

Queues: displayQueue and alarmQueue pass SensorData structs from SensorTask to DisplayTask and AlarmTask. modeQueue passes DisplayMode from InputTask to DisplayTask.

Event Groups: systemEvents holds three bits: EVENT_ACTIVE, EVENT_MOTION, and EVENT_ALARM. These are set and cleared by MotionTask, StateTask, and AlarmTask, and read by DisplayTask and StateTask.

3.4 Synchronization

A single mutex (logMutex) protects the shared Serial output. The competing tasks are all six tasks that call log_line(). The failure mode prevented is interleaved or corrupted log output when multiple tasks attempt to print simultaneously.

4. Implementation

4.1 Key Design Decisions

ESP-IDF over Arduino: The laboratory explicitly prohibits Arduino abstractions. Using native ESP-IDF APIs provided finer control over task scheduling, memory, and hardware peripherals.

vTaskDelayUntil() for Periodic Tasks: SensorTask and InputTask use vTaskDelayUntil() instead of vTaskDelay() to ensure consistent periodic execution without drift. This is critical for the sensor sampling period.

Queues over Global Variables: Passing data via queues ensures thread safety without complex locking and decouples producers from consumers.

Event Groups for System State: Event groups provide a lightweight, non-blocking way to broadcast system-wide state changes.

4.2 DHT22 Driver

The DHT22 driver implements the single-wire protocol with microsecond-level timing. A portENTER_CRITICAL section was initially used to protect the timing sequence, but this caused FreeRTOS assertion failures because the critical section was held for too long. The critical section was removed, and a retry mechanism was implemented in the SensorTask to handle occasional timeouts.

Known Limitation: In the Wokwi simulator, the DHT22 frequently returns ESP_ERR_TIMEOUT (263) due to timing inaccuracies in the simulation environment. This is a simulator limitation and does not reflect a defect in the firmware logic, which passes all unit tests.

4.3 OLED Display Driver

The SSD1306 driver was implemented from scratch using the I2C master API. A custom 5×7 font table is used to render text without external libraries. The display is owned exclusively by DisplayTask, which receives updates via displayQueue and modeQueue.

4.4 Alarm Logic

The temperature decision logic is separated into evaluateTemperature(), which returns an AlarmState enum. This function is pure and hardware-independent, allowing it to be tested on the native platform.

5. Verification and Testing

5.1 Unit Tests (13 Tests)

The following unit tests were implemented in test/test_main.cpp and pass on the native platform:

Category	Tests	Coverage
Temperature Alarm Logic	5	Below threshold, at lower threshold, normal, at upper threshold, above upper threshold
Display Navigation	4	Forward, reverse, forward wraparound, reverse wraparound
System State	4	ACTIVE with no timeout, ACTIVE with timeout, INACTIVE without motion, INACTIVE with motionAll

13 tests pass successfully with pio test -e native.

5.2 Wokwi Functional Tests (FT-01 to FT-10)

Test ID	Stimulus	Expected Result	Actual	Result
FT-01	Change temperature	Displayed temperature updates	Temperature displayed updated	PASS
FT-02	Change humidity	Displayed humidity updates	Humidity displayed updated	PASS
FT-03	Change light input	Light value changes	Light percentage changed on OLED	PASS
FT-04	Rotate encoder clockwise	Next page is selected	Mode changed from Temperature to Humidity	PASS
FT-05	Rotate encoder counterclockwise	Previous page is selected	Mode changed back to previous page	PASS
FT-06	Set temperature above 30°C	Alarm activates	Buzzer ON and "ALARM!" displayed	PASS
FT-07	Return temperature to normal	Alarm stops	Buzzer OFF	PASS
FT-08	Trigger PIR	System is ACTIVE	OLED powered on	PASS
FT-09	Allow inactivity timeout	System becomes INACTIVE	OLED powered off	PASS
FT-10	Trigger PIR while INACTIVE	System returns to ACTIVE	OLED powered on again	PASS

5.3 Fault Experiments

Three deliberate FreeRTOS fault experiments were performed:

Experiment 1 – Remove Blocking Delay: Removing `vTaskDelayUntil()` from `SensorTask` caused the task to run continuously without yielding, starving lower-priority tasks. The rotary encoder and OLED still functioned because higher-priority tasks preempted the `SensorTask`, but the experiment demonstrated the importance of blocking delays in cooperative multitasking.

Experiment 2 – Change Priority: Setting `DisplayTask` to priority 4 (highest) caused the encoder and OLED to become unresponsive. The slow I2C updates of the display task monopolized CPU time, starving the input task. This demonstrated the danger of assigning high priorities to slow tasks.

Experiment 3 – Remove Mutex: Commenting out the `logMutex` in `log_line()` did not visibly interleave logs because `ESP_LOGI` provides internal thread safety and the logging frequency was low. However, the experiment demonstrated that without a mutex, the `vsnprintf + ESP_LOGI` sequence could be preempted, potentially causing corrupted output under higher load.

6. Static Code Analysis

Static analysis was performed using `cppcheck` via `pio check`. The findings are summarized below:

Finding	File/Line	Cause	Resolution
unusedFunction	Multiple files	Functions registered via <code>xTaskCreate</code> or called cross-file; <code>cppcheck</code> cannot trace pointers	Documented as false positive
useStdAlgorithm	<code>oled_ssd1306.cpp:26</code>	Raw loop used instead of <code>std::find_if</code>	Accepted as intentional for embedded performance

The analysis produced no critical errors. All findings were either false positives or acceptable style deviations for an embedded system.

7. Engineering Discussion

7.1 Trade-offs

ESP-IDF vs Arduino: ESP-IDF provided greater control and learning value but required significantly more boilerplate code.

Custom OLED Driver vs Library: Implementing the SSD1306 driver from scratch was more work but eliminated external dependencies and gave full control over the I2C timing.

Queues vs Global Variables: Queues introduced some complexity but eliminated race conditions and made the data flow explicit.

7.2 Limitations

Wokwi Simulation: The Wokwi simulator cannot perfectly replicate the microsecond-level timing of the DHT22 sensor. This caused frequent `ESP_ERR_TIMEOUT (263)` errors during development.

No Physical Hardware Validation: All testing was performed in simulation. Real-world factors such as I2C bus capacitance, sensor noise, and power supply variations were not tested.

7.3 Debugging Challenges

The most significant challenge was the DHT22 driver. Initial implementations used a critical section that held interrupts disabled for too long, causing FreeRTOS assertion failures. After removing the critical section, the

driver worked correctly but occasionally timed out in Wokwi. This was ultimately accepted as a simulator limitation.

7.4 Alternative Approaches

An alternative to the event group would have been to use task notifications. An alternative to the custom OLED driver would have been to use the `esp_lcd` component from ESP-IDF. Both alternatives were considered but rejected in favor of the simpler, more educational implementation.

8. Conclusion

8.1 What Was Learned

This laboratory activity provided hands-on experience with:

FreeRTOS task creation, scheduling, and priority assignment.

Inter-task communication using queues and event groups.

Shared resource protection using mutexes.

Hardware driver development using native ESP-IDF APIs.

Unit testing of hardware-independent logic.

Static code analysis and interpretation of findings.

8.2 Future Improvements

Migrate the project to physical hardware and calibrate the LDR for lux measurement.

Implement a web server using ESP32 Wi-Fi for remote monitoring.

Add SD card logging for historical data analysis.

Improve the DHT22 driver with more robust error handling.

Prepared by: Abigil Queen Casas

Date Submitted: September 24, 2026